MovEazy — Architecture, Workflow & Operations Guide

Document version: 1.0 · **February 2026** · Repository: MOVEASY-WEBSITE (React SPA + Firebase)

1. Executive summary

MovEazy is a **single-page application (SPA)** served as static files. The browser runs the UI; **Firebase** provides authentication, Firestore (document database), file storage, optional Cloud Functions, and hosting. There is **no traditional Node/Express API** in the main user path—data access is **direct from the client to Firestore** under **security rules**, which enforce who can read or write what.

2. Technology map

Layer	Technology	Role
Language (UI)	JavaScript (ES modules) + JSX	React components and client logic
UI framework	React 19	Components, state, effects
Routing	React Router v7	Public pages; role routes for admin, seller, customer, staff CRM
Build	Vite 8	Dev server, HMR, production bundle to `dist/`
Styling	Tailwind CSS 3 + inline styles (CRM)	Layout and marketing
Maps	Leaflet + React-Leaflet	Interactive listing map
Animation	GSAP, Framer Motion	Marketing motion
Auth & data	Firebase JS SDK v12	`auth`, `firestore`, `storage`, `functions`
Errors (client)	Sentry	Optional client error capture
Backend (optional)	Cloud Functions (Node 20)	Triggers, Admin SDK—separate deploy from hosting
Infrastructure	Firebase Hosting	Static hosting; SPA fallback to `index.html`
Authorization	`firestore.rules`, `storage.rules`	Server-side enforcement for data access
CI	GitHub Actions	Build / test / deploy
Tests	Vitest, Playwright	Unit and e2e

3. End-to-end workflow

3.1 First visit → marketing

1. User opens the hosted site; **Firebase Hosting** serves `index.html` and assets.
2. React starts from `src/main.jsx`; `App.jsx` routes to `/`, `/services`, `/map`, etc.
3. Most marketing and map discovery does not require login.

3.2 Sign-up / sign-in

1. **AuthContext** listens to Firebase Auth (`onAuthStateChanged`).
2. Email/password and Google sign-in (per app code).
3. Profiles and roles in Firestore (`userProfiles`, `userRoles`, etc.).
4. Admin allowlist via env `VITE_ADMIN_EMAILS` may supplement Firestore roles.

3.3 Role-based areas

- `/customer`, `/seller`, `/admin`: UI gated by role; Firestore rules must match.
- `/crm`: Staff (`admin`, `sub_admin`, `consultant`) — leads, tasks, notifications.

3.4 Map & listings

`/map` uses Leaflet; listings load from Firestore. Public vs private fields (e.g. broker phone) depend on **rules** and client sanitization helpers.

3.5 Staff CRM

- Data: `crmLeads`, `crmTasks`, `notifications` via `firestoreStore.js`.
- Sub-admins/consultants: scoped to `assigneeEmail`; admins see all.
- Optional TSV paste import; `extraFields` for legacy sheet columns.

3.6 Deploy

`npm run build` → `dist/`. `firebase deploy --only hosting` (and often `firestore:rules`). Functions deploy separately.

4. Security model (short)

- **Authentication:** Firebase Auth.
- **Authorization:** Firestore & Storage **rules** — primary enforcement.
- **App Check:** Optional reCAPTCHA v3 to reduce API abuse.
- React role checks improve UX only; they do not replace rules.

5. Observability & quality

- Sentry and client logging helpers where configured.
- Vitest / Playwright for regression; ESLint for style—full-repo lint may include legacy issues.

6. Skills for this stack (2026)

Core: Modern JavaScript + React; Firebase Auth + Firestore + rules; Git + CI.

Strongly recommended: SPA threat modeling; performance basics; e2e smoke tests for auth and money paths.

7. “Vibe coding” and common failure modes

> **Context:** AI-assisted “vibe coding” often ships UI fast but under-invests in security, errors, and ops. For this Firebase SPA, the highest-risk gaps are:

Area	Risk	Mitigation
Authorization	UI hides actions but rules stay permissive	Rules review per collection; test as each role
Authentication	Session / verification edge cases	Test Google vs password; document flows
Data layer	Silent `permission-denied`	User-visible errors + logging
Error logging	Empty catch blocks	Sentry or structured client logs
Cloud Functions	Hosting OK, callables broken	Separate health checks; monitor logs
Environment	Missing `VITE_*` at build	Checklist + staging deploy
Indexes	New queries fail until indexes exist	Use Firestore console links from errors

Bottom line: Production-grade work requires **verification**—treat the client as untrusted, align rules with product, and test failure paths.

8. Reference paths in repo

- `firebase.json` — hosting, Firestore, storage, functions
- `firestore.rules` — data authorization
- `src/App.jsx` — routes
- `src/lib/firebase.js` — Firebase init, App Check
- `src/lib/firestoreStore.js` — data access helpers
- `docs/DEPLOYMENT.md` — deployment notes

- `docs/crm-all-header-rows.txt` — CRM sheet headers

Source markdown: `docs/MOVEAZY_ARCHITECTURE_AND_WORKFLOW.md` · To save as PDF: open this HTML in a browser → Print → Save as PDF.